

AIOps Module 3 – Assignment 2

Ramana Gopal Kodali & DA24B010

Q1. Single-Stage vs. Multi-Stage Docker

Q1.1 – Naive Docker Image

I have built the spam-detection API using a single-stage Dockerfile and checked that the container served both `/healthz` and `/predict` correctly. Then I have downloaded the images and found that The Docker-reported disk usage was **2.31 GB**, while the image/content size was approximately **598 MB**.

Q1.2 – Multi-Stage Build

Then I implemented the second step in the Q1 and changed the Dockerfile to a multi-stage build. The builder stage uses `python:3.12`, while the final application stage uses `python:3.12-slim`. The final Docker-reported disk usage was **742 MB** and the image/content size was approximately **159 MB**.

$$\text{Disk reduction} = \frac{2.31 - 0.742}{2.31} \times 100 = \mathbf{67.88\%}$$

$$\text{Image/content reduction} = \frac{598 - 159}{598} \times 100 = \mathbf{73.24\%}$$

Q1.3 – Why the Multi-Stage Image is Smaller

The builder stage contains the full Python image and the dependency installation environment, but these build-time contents are not copied into the final image. Only the installed dependencies from `/install` and the application files are copied into the `python:3.12-slim` runtime image. Thing is I also used `pip --no-cache-dir`, so pip's download cache is not retained.

Q2. Multi-Container Orchestration with Docker Compose

Q2.1 – Redis Cache

As given in the question I have extended the API with Redis caching. For each input text, the API creates a SHA-256 hash and checks Redis before running the model. And On a cache MISS, the model performs the prediction and the result is stored using `SETEX`. On a cache HIT, the stored label is returned directly.

The cache TTL used in my implementation is **300 seconds (5 minutes)**.

Q2.2 – Docker Compose

The Compose file contains two services: the API and `redis:7-alpine`. The API connects to Redis using the Compose service name as its hostname. I verified the stack using `docker compose ps`, the `/healthz` endpoint and Redis connectivity.

Q2.3 – Cache-Hit Evidence

For the same input, my benchmark file have produced:

Request	Time
Cache MISS	28.172 ms
Cache HIT	2.463 ms

Both requests returned the same spam label.

$$\text{Speedup} = \frac{28.172}{2.463} = \mathbf{11.44\times}$$

The measured latency reduction was **91.26%**. This gives direct timing evidence that the repeated request was faster when the result was served from Redis.

Q2.4 – Compose vs. Kubernetes

Docker Compose allows multiple related containers, such as the API and Redis in this task, to be defined, networked and started together on one Docker host. Kubernetes is intended for cluster-level orchestration and adds features such as scheduling across nodes, Services, Jobs, self-healing and rolling updates.

Q3. Kubernetes Indexed Job: Parallel Data Validation

Q3.1 – Parallelism and Job Design

I made a .py which generated eight deterministic CSV shards, with each shard containing **20 rows**. Each Indexed Job Pod processes one shard using `JOB_COMPLETION_INDEX`.

The cluster has two nodes with two allocatable CPUs each:

$$2 \times 2 = 4 \text{ CPUs}$$

Each validation Pod requests and limits **1 CPU**. Therefore the resource-based maximum is:

$$\frac{4 \text{ CPUs}}{1 \text{ CPU per Pod}} = 4 \text{ Pods}$$

I therefore used `completions: 8` and `parallelism: 4`. The Job also uses `completionMode: Indexed` and `restartPolicy: Never`. The Downward API provides `POD_NAME` and `NODE_NAME` so that the Pod and node can be identified.

Q3.2 – Job Execution

The Job completed all **8/8 completions** with zero failures. The captured `kubectl get pods -o wide` output showed two validator Pods running at the same time in the observed execution, followed by subsequent Pods. Thus, `parallelism: 4` was the configured maximum, but the observed paralisation is 2 Pods simultaneously. The Pods were limited by the available CPU capacity on the nodes. So, although parallelism was set to 4, only 2 Pods ran at the same time.

Q3.3 – Collecting the Results

The results were collected through the Kubernetes API by reading the completed Pods' logs using `CoreV1Api, list_namespaced_pod()` and `read_namespaced_pod_log()`. I did not use a shared volume because the Minikube multi-node setup uses node-local storage, so a file written on one node cannot be assumed to be directly available on another node.

Shard	Rows	Invalid Rows
0	20	3
1	20	4
2	20	5
3	20	3
4	20	4
5	20	5
6	20	3
7	20	4
Total	160	31

Q3.4 – If 6 CPUs Were Available

With three nodes having two CPUs each, the cluster would have six allocatable CPUs. Since each validator requests one CPU, I would use `parallelism: 6`, allowing up to six Pods to execute concurrently. The remaining two completions could then run after CPU capacity becomes available.

Q4. Kubernetes Deployment: Self-Healing and Rolling Updates

Q4.1 – Deployment and Service

In this I have deployed the spam-detection API using a Kubernetes Deployment with **2 replicas**. The Deployment specifies CPU requests and limits and a readiness probe for `/healthz`. A NodePort Service exposes the API. The final Deployment state showed **2/2 replicas ready**.

Q4.2 – Self-Healing

To test self-healing, I manually deleted one running Pod using `kubectl delete pod <pod-name>`. Kubernetes automatically created a replacement Pod, which became Ready and restored the Deployment to two replicas.

The Deployment manages a ReplicaSet, which maintains the desired number of Pods. When the actual number falls below the desired replica count, the ReplicaSet creates a replacement Pod.

Q4.3 – Rolling Update

I have made a visible version change to the API's `/healthz` response, built the changed application as a new image tag and updated the Deployment to use the new image with tag "v2". I verified the update using `kubectl rollout status` and `kubectl rollout history`, and confirmed the new version through the health endpoint.

If this API had been deployed as a Job, its Pods would be intended for finite run-to-completion work rather than continuously serving requests. A Job would therefore not maintain two long-running API replicas or provide the Deployment-style rolling update behavior used here.

Q4.4 – Deployment vs. Job

The Deployment fits the spam API because the API is a long-running service that needs a maintained number of replicas, self-healing and rolling updates. The Indexed Job fits the signup validation workload because it is a finite batch task where each shard is processed and the Pods terminate after completing their assigned work.